Lecture 1 · Foundations / SDLC / Git

SDLC: พัฒนาระบบ ไม่ใช่แค่เขียนโค้ด
Software engineering: แก้ปัญหาผู้ใช้ด้วยระบบที่ทดสอบได้ ปลอดภัยและดูแลต่อได้; โค้ดรันได้ ≠ งานผ่านการยอมรับ Planning → Requirements → Specification → Design → Implementation → Testing → Deployment → Maintenance **Planning**: เหตุผล/ความเป็นไปได้/ขอบเขต/ความเสี่ยง; **Req**: ต้องการอะไร; **Spec**: พฤติกรรมที่ตกลง; **Design**: ทำอย่างไร; **Test**: หลักฐาน; **Deploy**: build/config/migrate/monitor/rollback; **Maintain**: แก้ไขและปรับระบบ **Feasibility**: เทคนิคพร้อม? คุ้มทุน? ใช้งานในองค์กรได้? ทันเวลาและมีทรัพยากรพอ? ไม่ใช่ดูแค่โค้ดรันได้

Process models / Agile values

Model	จุดเด่น / ข้อควรระวัง
Waterfall	แผนและ sign-off ชัด; เหมาะกับ req คงที่ แต่ feedback ช้า เปลี่ยนแพง
Incremental	ส่ง vertical slice ที่ใช้ได้ทีละส่วน; คุม architecture และ integration
Iterative	ปรับ solution เดิมซ้ำตาม feedback
Scrum	sprint goal, timebox, increment; ตรวจสอบและปรับตัว
Kanban	งานไหลต่อเนื่อง; จำกัด WIP วัด cycle time/throughput

Agile ให้ความสำคัญมากกว่า: คนและปฏิสัมพันธ์ > process/tools; ซอฟต์แวร์ที่ทำงานได้ > เอกสารละเอียด; ร่วมมือกับลูกค้า > ต่อรองสัญญา; ตอบรับการเปลี่ยนแปลง > ยึดแผน ด้านขวายังมีคุณค่า **Vertical slice**: UI + API + DB + test ใช้ได้ครบเส้นทาง; ไม่ใช่ทำ DB อย่างเดียวแล้วค่อยต่อ UI **DoD**: ผ่าน AC/tests/review; ไม่มี defect หรือช่องโหว่ร้ายแรง; docs ตรง; merge/release ตามกติกา

AI engineering / SDD + TDD
Agent: observe → plan → tool/edit/test → inspect → adjust; ต่างจาก chatbot ที่แค่ตอบข้อความ **Prompt**: บริบท + req/contract + ข้อจำกัด + ขอบเขตไฟล์ + tests + หลักฐาน; วางแผน แบ่งงาน ตรวจ diff/ผลจริง คนรับผิดชอบเป้าหมาย การยอมรับและความเสี่ยง; คำว่า "ผ่าน" จาก agent ไม่ใช่หลักฐาน ระวัง hallucination, regression, secrets, dependencies และ docs ที่ล้าสมัย

Architecture / Git พื้นฐาน
React UI → HTTP API/Express → service/BR → Prisma ORM → PostgreSQL; UI ไม่ต่อ DB โดยตรง **Working tree** แก้ไฟล์ → `add` เลือก snapshot เข้า **index** → `commit` บันทึกใน local → `push` ส่งไป remote Issue → branch → commit → PR → review → staging → main; feature แยกงาน; staging รวม/ทดสอบ; main คืองานที่ยอมรับ; README บอก setup/run/test/env/กติกา

```
git status; git diff; git diff --cached
git log --oneline --graph --decorate --all
```

ไม่เก็บ .env, credentials, node_modules หรือ temp; ใช้ .env.example และระบุขั้นไฟล์ตอน `git add`

Lecture 2 · Requirement Engineering

Business: เป้าหมาย/คุณค่า; **User/URS**: ผู้ใช้ต้องการอะไรและทำไม; **System/SysRS**: รวม HW, interfaces, บุคลากร/กฎระเบียบ; **Software/SRS**: พฤติกรรมและข้อจำกัดของซอฟต์แวร์

ชนิด	คำถาม / ตัวอย่าง
FR	ทำอะไร? Requester สร้าง Ticket ได้
NFR	ดีแค่ไหน? response p95 ≤ 2 s เมื่อรับ load ที่ระบุ
BR/domain	กฎควบคุมพฤติกรรม; ไม่เก็บ 3 ครั้ง/ภาค
Constraint	ข้อจำกัด; ต้องใช้ React/ที่ตั้ง server

FR/NFR/BR เป็นชนิด ไม่ใช่ระดับ user/system; กฎหนึ่งผูกกับหลาย FR ได้ ไม่ต้องสร้าง BR ให้ทุก pixel **NFR**: performance, security, usability, reliability, maintainability; ระบุตัวชี้วัด เงื่อนไขและเกณฑ์ผ่าน

RE process / Quality checklist
Elicit/analyse → specify → validate → manage changes; หา stakeholder/goal → สัมภาษณ์/สังเกต/อ่านเอกสาร → บันทึก → ยืนยัน → จัดลำดับ **Fact**: ระบุแล้ว; **Assumption**: ยังไม่ยืนยัน; **Question**: ต้องถาม; **Constraint**: จำกัดทางเลือก; แยก in/out/later scope **Quality**: ชัดเจน ไม่กำกวม มีเรื่องเดียว จำเป็น ครบ สอดคล้อง ทำได้จริง ทดสอบและย้อนรอยได้; ระบุ actor/action/object/condition **Validation**: ตรงความต้องการ สอดคล้อง ครบ เป็นจริงได้ ตรวจสอบได้; ใช้ review/prototype/derive tests. **Change control**: เหตุผล/ผู้รับผิดชอบ/ผลต่อเวลา-ต้นทุน-ความเสี่ยง/ข้อสรุป/รุ่น

Proposal 8 ช่อง / Evidence chain
Problem · Users · Solution · Expected value · Scope · Risk · Effort · Evidence of completion แยกสิ่งที่มี/ขาดพร้อมหลักฐาน; ข้อเสนอเพิ่มต้องติดป้าย ไม่เขียนเป็นข้อเท็จจริง Req ID/source → feature/design → issue/code → AC/test → PR/release → demo; ต้องชี้ไฟล์และผลจริง **AC**: เกณฑ์เฉพาะ feature; **DoD**: เกณฑ์เสร็จร่วม; **DoR**: พร้อมเริ่ม ≠ เสร็จ.
MoSCoW: Must / Should / Could / Won't (รอบนี้)

วิเคราะห์ requirement ให้ตอบเป็นเหตุผล

ปัญหา	ต้องซื้ออะไร / แก้อย่างไร
กำกวม	"quick/easy" ไม่มีเกณฑ์ → ตกลง metric/load/threshold
ไม่ atomic	หลาย actor/หลายเป้าหมาย → แยกข้อ แต่คง trace กลับต้นฉบับ
ไม่ครบ	ขาด precondition/output/error/สิทธิ์ → ถามแต่ละสถานะ
ขัดแย้ง	เช่น limit 3 กับ 4 → ให้ผู้มีอำนาจตัดสิน ไม่เลือกเอง
ตรวจไม่ได้	แปลงคำกว้างเป็นผลสังเกตได้ พร้อมวิธีวัด
ทำไม่ได้	ตรวจเทคนิค/เวลา/ทรัพย์; เจรจาขอบเขต ไม่รับปากลอย ๆ
ย้อนกลับไม่ได้	ใส่ ID/source/owner แล้วโยง AC/design/test

Elicitation: ใครทำ? เมื่อไร? ข้อมูลเข้า/ออก? กฎ/สิทธิ์? ข้อยกเว้น? นอกขอบเขต? ใช้ interview/observation/docs/prototype ตรวจความเข้าใจร่วม **BR vs FR**: "ไม่เกิน 3 ครั้ง/ภาค" คือ BR; "ระบบปฏิเสธครั้งที่ 4" คือ FR ที่บังคับกฎนั้น; NFR ระบุระดับคุณภาพของการทำงาน AC ตัวอย่าง: Given สมัครแล้ว 3 ครั้งในภาคนี้; When ส่งครั้งที่ 4; Then ปฏิเสธและจำนวนยังเป็น 3. ต้องตกลงก่อนว่าการยกเลิกนับรวมไหม

Lecture 3 · Specifications / SDD / Tests

SRS → SDS → STS
SRS: ต้องการอะไร (FR/BR/NFR); **SDS**: ออกแบบอย่างไร; **STS**: ตรวจอย่างไร (cases/data/expected/evidence) SRS → feature inventory → req-feature map → system SDS → sprint vertical slice → feature contract → implement → test/review → integrate → main **System SDS**: architecture/stack/modules, data, auth, API/error conventions, UI design system, security/performance/logging **Feature SDS**: purpose/scope, linked req, workflow/state, BR, data/API/UI, permissions, validation/errors, AC/tests **Contract 7**: inputs → validations → behavior → persistence → side effects → errors → outputs ตัวอย่าง invoice: รับ header/lines → ตรวจ required/qty → คำนวณ total → บันทึก transaction เดียว → ออก receipt หลัง commit → line ผิดต้อง rollback ไม่เหลือ header → คืน invoice ID/total/ผลบันทึก

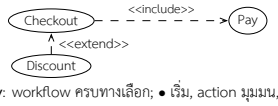
Tests / Red-Green-Refactor
RED: test ล้มเหลวด้วยเหตุผลที่ตั้งใจ → GREEN: เขียนโค้ดเท่าที่จำเป็นให้ผ่าน → REFACTOR: ปรับโครงสร้างโดย tests ยังผ่าน **AAA**: Arrange เตรียมข้อมูล → Act เรียกใช้ → Assert ตรวจผล; ครอบคลุมปกติ/ผิดเงื่อนไข/ขอบเขต/error/สิทธิ์; ต้องจับ defect ได้ ไม่เขียนตาม implementation อย่างเดียว **STS row**: test ID + Req/AC → precondition/data → steps/input → expected UI/API/DB → file + final result Boundary example: คะแนน 0-100 → ทดสอบ −1/0/100/101; 0 และ 100 ต้องผ่าน ส่วน −1 และ 101 ต้องถูกปฏิเสธ

ระดับ/ชนิด	ตรวจอะไร
Unit	function/class/component; แยก deps ด้วย mock
Integration	modules/API/DB/interfaces ทำงานร่วมกันจริง
System	ระบบรวมตรง SRS ทั้ง FR/NFR
E2E	เส้นทางผู้ใช้งาน UI → API → DB/external
UAT	ผู้ใช้/ธุรกิจยอมรับตาม need/AC และ sign-off
Regression	ทดสอบของเดิมหลังแก้ไข; ใช้ได้ทุกระดับ
Smoke	เปิดระบบและใช้งานหลักได้หลัง setup/release
Special	load/performance, security, compatibility, recovery

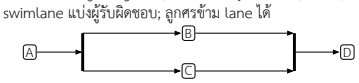
Verify: ทำตรง spec? **Validate**: ตรงความต้องการผู้ใช้? coverage สูง ≠ tests ดี **Sync**: ผู้เรียกรอผล; **Async**: รับผลภายหลังผ่าน event/callback; ระบุวิธี retry และ idempotency (เรียกซ้ำได้ผลต่อระบบเท่าเดิม) ใน contract เช่น retry ด้วย key เดิม ต้องไม่ตัดเงินซ้ำ **BDD**: เริ่มจากพฤติกรรมด้วย Gherkin (outside-in); **TDD**: ให้ tests นำการออกแบบ (inside-out); ใช้ร่วมกันได้

Lecture 4 · UML / Feature Design / UI

Use case: เป้าหมายของ actor ภายใน system boundary; วงรี = goal; actor association เส้นทึบ ไม่มีหัวลูกศร include: base → ขั้นตอนบังคับ (เส้นประ); extend: กรณีเสริม → base (เส้นประ + condition); generalization: child → parent (สามเหลี่ยมโปร่ง)



Activity: workflow ครบทางเลือก; ● เริ่ม, actionมุมมน, ◇ decision/merge + [guard], bar = fork/join ขนาน, ⊙ จบ; swimlane แบ่งผู้รับผิดชอบ; ลูกศรข้าม lane ได้



fork: ทำ B/C ขนาน; join: รอครบทั้งคู่

Sequence: scenario หนึ่งใน workflow; participant/lifeline อยู่ด้านบน เวลาลงล่าง; call เส้นทึบ/return เส้นประ; activation คือช่วงทำงาน; alt [condition], opt, loop; แสดงระบบภายนอกและผลข้างเคียง **State**: สถานะของ object เดียว; event [guard] / effect; ระบุจุดเริ่ม/จบและ transition ที่อนุญาต; อย่าสับสน action กับ state



Class (Lecture 4; Sample ระบุ Final): name/attributes:type/operations; association + multiplicity (1, 0..1, 0..*); inheritance ชี้ parent; composition ◆ ที่ whole (part มีวงจรชีวิตผูกกัน); aggregation ◇ ผูกหลวม; dependency เส้นประ **Scenario**: basic = ปกติ; alternative = ทางเลือกที่ถูกต้อง; exception = ผิดพลาด; Activity ≠ Sequence; Use Case ≠ flowchart

Generic UI: List / Form / LoV
List: title/user/role, search/filter/sort, columns ตาม req, select/open/create, pagination, empty/loading/error; **Form**: header + line items + total/status + controls **สัญลักษณ์สอบ**: * นำหน้าช่องบังคับ; textbox แก้ได้; ขีดเฉียง = read-only/calculated; เลือก FK ด้วย LoV; รายการย่อยมี add/remove และ validation **ตามสถานะ**: Draft แก้/Save/Submit ได้; Done ล็อก; แสดง pending/failure/retry/confirmation ตาม contract; header field ≠ line-item field **Design system**: สี/ตัวอักษร/ระยะ/components/grid/breakpoints/contrast/keyboard/focus; ตารางยาวอาจเลื่อนแนวนอน ไม่เติม split payment/cancel/refund หากโจทย์ไม่ให้; ทุก field/button/transition ต้องโยง req และสิทธิ์ได้

Case patterns จาก Lecture 4 / review

Case	สิ่งที่ต้องรักษาเมื่อวาด UML/UI
POS	Cashier/Customer; รายการสินค้า/ยอด; cash/card; card ไม่ผ่าน → retry/cash/cancel; สำเร็จ → receipt/stock
Hotel	Guest ค้นวัน/เมือง เลือกหลายห้อง/ประเภท จ่าย KBank → ลดห้องว่าง/ยืนยัน/email ถึง Guest และ Hotel; Hotel Admin ดูแลโรงแรมตน
Ticket	Customer สร้าง Requested พร้อม action อัตโนมัติ; CCE/Programmer เพิ่ม action ตาม state; action แก้/ลบไม่ได้; latest status/due อัปเดต; ปิดแล้วห้ามเพิ่ม
Grades	Instructor เลือก semester/course/sections ดึงรายชื่อ/กรอกเกรด; ครบ → Ready → submit → Submitted; อัปเดต Transcript/email; ใช้ SSO/Registration/Student Info/Email ภายนอก

ก่อนส่งรูป: ทุกเส้นโยง req ได้; guards ครบ; error กลับจุดแก้; success มี side effects ตามโจทย์; ไม่แต่ง state/สิทธิ์

Lecture 5 · Security Engineering

Security landscape / CIA
Actor: hacker/user/insider/client ที่ผิดพลาด; **Asset**: password, ticket, DB; **Attack surface**: form/API/upload/admin; **Trust boundary**: browser → server, app → DB C: Confidentiality ป้องกันการเห็นข้อมูลโดยไม่มีสิทธิ์; I: Integrity ป้องกันการแก้ไขโดยมิชอบ; A: Availability ผู้มีสิทธิ์ยังใช้ระบบได้ **Secure by design**: least privilege, defense in depth, fail safe/default deny, ออกแบบเรียบง่าย แบ่งส่วน ลดจุดโจมตี; ค่าเริ่มต้นปลอดภัยและยังใช้งานได้

Authentication ≠ Authorization
AuthN: เป็นใคร? ตรวจตัวตน/บัญชี active/หลักฐาน; **AuthZ**: ทำได้ไหม? subject + action + object + state; ตรวจที่ backend ทุก request **RBAC**: role → operations; **ABAC**: ตรวจ attributes ของผู้ใช้/resource/context; **ACL**: รายการสิทธิ์ต่อ resource; ownership ใช้ ID จากการยืนยันตัวตน ไม่เชื่อ userId ที่ client ส่งมา **Password**: slow hash + salt เฉพาะแต่ละรหัส; ไม่เก็บ/บันทึก plaintext. Base64 แค่แปลงรูปแบบ ไม่ใช่ encryption; encryption ถอดคืนด้วย key; hash ทางเดียวใช้ตรวจเทียบ **Session**: login → server สร้าง session → cookie → หมดอายุ/logout แล้วเพิกถอน; เข้าหน้าป้องกันซ้ำต้องถูกปฏิเสธ; current-user คืน profile ไม่คืน secret **JWT**: signed claims ไม่ใช่ encryption; ตรวจ signature/expiry; bearer ใช้ได้เมื่อถือ token; ออกแบบ logout/revocation เอง; access token อายุสั้น, refresh token ต้องปกป้อง/เพิกถอนได้ **API key**: secret ของ client/service; **OAuth2**: มอบสิทธิ์ให้แอปอื่น; **OIDC**: OAuth2 + identity/ID token; **SSO**: เข้าใช้หลายระบบด้วยการ sign-in ครั้งเดียว

OWASP / Vulnerability / Control

ช่องโหว่	การป้องกัน / การทดสอบเชิงลบ
Broken access	ตรวจ role/owner/state ที่ backend; เรียก API ข้ามคน/role → ปฏิเสธ ไม่รั่วข้อมูล
Auth failure	error กลาง, rate limit, salted hash, expiry; ลอง wrong/inactive/logout แล้วเข้าต่อ
SQLi	parameterized query/ORM; allowlist ชื่อ field; `' OR '1'='1` ต้องเป็นข้อมูล ไม่ใช่ SQL
XSS	encode output, sanitise rich text; ไม่ใช้ HTML ที่ไม่ปลอดภัย; payload ต้องไม่รัน
CSRF	browser ส่ง cookie ข้าม site; ใช้ token/SameSite/Origin check; ปฏิเสธ forged write
SSRF	server ดึง URL จาก input; allowlist ปลายทาง/บล็อก internal/metadata/loopback
Upload	server ตรวจชนิด/ขนาด/เนื้อหา; เก็บแบบรันไม่ได้/ดาวน์โหลดปลอดภัย; script ปลอม PDF ต้องไม่รัน
Config/secrets	HTTPS, secure cookie, CORS เคร่งครัด; ไม่เผย debug/.env; เปลี่ยน secret ที่รั่ว
Deps	patch/audit packages; จำกัดสิทธิ์ DB/network; ปกป้อง backups

ซ่อนปุ่มไม่ใช่การควบคุมสิทธิ์; frontend validation ช่วย UX; backend validation คุม trust boundary **Status**: 400 ข้อมูลผิด; 401 ยังไม่ยืนยันตัวตน; 403 ไม่มีสิทธิ์; 404 ไม่พบ/ปิดบังตามกติกา; 409 ขัดแย้ง; 500 ข้อผิดพลาดกลาง; log correlation ID ไม่ใช่ password/token **Reject**: ตรวจ error/status + business data ไม่เปลี่ยน + ไม่มีผลข้างเคียงที่ห้าม; audit log ยังเกิดได้

Security check: ทำตรง ๆ แล้วต้องเห็นอะไร

ทดสอบ	ผลที่ต้องตรวจตาม policy/contract
AuthN/gate	valid ผ่าน; wrong/inactive/ไม่ login ถูกปฏิเสธ; password gate ต้องคุม API ตรง
ผิด role	เรียก endpoint ตรงไม่ได้; ไม่มี business write
ข้าม owner	A เปลี่ยน ID ไปของข้อมูล B → ไม่เผยข้อมูล/จำนวน/ไฟล์
logout/expiry	ใช้ session เดิมซ้ำไม่ได้ แม้เปิด URL/API ตรง
input/duplicate	error ปลอดภัย; DB ไม่ค้างครึ่งชุด; ข้อมูลเดิมไม่เสีย
log/config	ไม่พบ token/password/stack trace ใน response หรือ log ที่เปิดเผย

ลำดับตรวจ: AuthN → AuthZ (role/owner/state) → validation → business action → persistence; ตรวจทุก request ไม่เชื่อ userId/role จาก client **Defense in depth**: ต่อให้ไฟล์หลุด validation ต้องยังรันไม่ได้และอ่านข้าม owner ไม่ได้

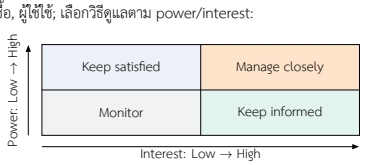
Lecture 6 · Project Initiation

Initiate → Plan → Execute → Close; เป็นวงจรโครงการ ส่วน SDLC อยู่ภายในโครงการ **Initiation 6**: goals, scope, deliverables, success criteria, stakeholders, resources → charter → อนุมัติ → วางแผน **Goal**: ผลที่ต้องการ;

Deliverable: สิ่งส่งมอบ; **Success criteria**: หลักฐานว่าสำเร็จ; launch = ส่งมอบ, landing = วัดผล **SMART**: Specific, Measurable, Attainable, Relevant, Time-bound; ลด median first-response 20% ภายในภาคนี้ด้วย routing อัตโนมัติ

OKR: Objective = ทิศทาง; KRs = ผลที่วัดได้; ทำทายและเชื่อมองค์กร → แผน → โครงการ; KR ไม่ใช่รายการงาน

ROI=(Gain-Cost)/Cost ×100%; รวมต้นทุนครั้งเดียว/ประจำในช่วงเวลาเดียวกัน; พิจารณาคุณค่าที่ไม่เป็นตัวเงินแยก G=10,000; C=6,000+25×12=6,300 → ROI=58.7% **Scope**: ตกลง in/out, เวลา/ทุน/ทรัพยากร/ข้อจำกัด; **Creep**: ลูกค้าเพิ่มงานหรือทีมเดิมเอง (gold-plating); เปลี่ยน → วิเคราะห์ผล → แลกเปลี่ยนขอบเขต → อนุมัติ → บันทึก **Triangle**: scope/time/cost; เมื่อตรึงด้านหนึ่งต้องปรับด้านอื่น ไม่รับงานเพิ่มโดยไม่ประเมิน **Stakeholders**: sponsor อนุมัติทุน, PM คุมแผน, ทีมทำ, ลูกค้าซื้อ, ผู้ใช้ใช้; เลือกวิธีดูแลตาม power/interest:



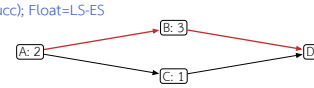
Charter: เหตุผล/เป้าหมาย/ขอบเขต/สิ่งส่งมอบ/เกณฑ์สำเร็จ/ทีม/ทรัพยากร/ทุน/กำหนดการ/สมมติฐาน/ความเสี่ยง/ผู้อนุมัติ

เกณฑ์ก่อนเริ่ม: เปิดเวิร์ก = deliverable; ลดเวลารอ = goal; criteria ต้องมี baseline/target/deadline ที่ตกลง

Lecture 6 · Project Planning

WBS / Schedule / Responsibility
WBS: project → milestone/deliverable → task → subtask; แยกแบบ top-down/bottom-up. **Milestone**: จุดสำเร็จ duration = 0; task มี owner และ duration **Gantt**: WBS no./task/owner/start/due + แถบเวลา; ทำขนานได้เมื่อ dependency และทรัพยากรพร้อม; roadmap เน้น goal/milestone ภาพรวม **Critical path**: เส้นทาง dependency ที่ยาวสุด กำหนดวันจบ; float คือเวลาที่เลื่อนได้โดยไม่กระทบวันจบ;

CP มี float = 0 เมื่อ deadline เท่ากับ earliest finish เริ่มเวลา 0: EF=ES+d; ES=max(EF ของ pred); LS=LF-d; LF=min(LS ของ succ); Float=LS-ES



Example: A → B → D = 7 days (CP)
A → C → D = 5 days; C float = 2

Task	ES	EF	LS	LF	Float
A	0	2	0	2	0
B	2	5	2	5	0
C	2	3	4	5	2
D	5	7	5	7	0

นับวันที่แบบ inclusive: Due = Start + d − 1 วันทำงาน; งานถัดไปเริ่มวันทำงานถัดไป; ไม่นับ weekend และห้ามคนเดียวทำงานข้อนเกินกำลัง **Effort**: person-hours/days ≠ **duration**: เวลาที่งานใช้; Brooks: เพิ่มคนในงานที่ล่าช้าอาจยิ่งช้า PERT $E=(O+4M+P)/6$; SD = $(P-O)/6$; O ดีสุด, M เป็นไปได้มากสุด, P แย่สุด; variance=SD^2; รวมงานบนเส้นทางที่พิจารณา; เมื่อ estimates อิสระ $SD_{total}=\sqrt{\sum_i SD_i^2}$ **Buffer**: task buffer เผื่อ delay ที่คุมไม่ได้ เช่น รออนุมัติภายนอก; project buffer เผื่อรวมทั้งโครงการ; ไม่ใส่ทุก task ซ้ำซ้อน **RACI**: R ลงมือ, A รับผิดชอบผล, C ปรึกษาก่อน, I แจ้ง; ให้ A ชัดคนเดียว/งานและมี R; ไม่เกี่ยวใช้ "–"; ระวัง R กระจุกตัว **Risk**: identify → analyse (probability × impact) → prioritise → owner → respond → monitor; แผนรับมือต้องมีเจ้าของ; issue คือเกิดแล้ว **Kick-off**: เป้าหมาย/ขอบเขต/บทบาท/tools/process/สิ่งต้องทำ; charter = ทำไม, plan = อย่างไร/เมื่อไร, risk register = อะไรอาจผิด

Lecture 7 · Execution / Agile / Scrum

MVP: ชิ้นเล็กสุดที่ทดสอบคุณค่าได้ → build-measure-learn → pivot/persevere; ไม่ใช่ phase 1 ที่ยังใช้ไม่ได้ **Pair**: driver เขียน/navigator ตรวจ สลับกัน; **DevOps**: dev + ops ร่วมดูแล delivery ใช้ automation/feedback; Water-Scrum-Fall ยังติด batch/approval ที่หัวท้าย **PO**: คุณค่า/วิสัยทัศน์/เรียง backlog/ตัดสิน req/ยอมรับ/ส่งมอบ; **SM**: อำนวยความสะดวก คุมเวลา แก้อุปสรรค ไม่สั่งแบ่งงาน/ตัดสิน product แทน PO; **Developers**: จัดการตนเอง ข้ามทักษะ เลือกวิธีทำ/คน/capacity **3 artifacts**: product backlog, sprint backlog, Done increment; **5 events**: Sprint, Planning, Daily, Review, Retrospective; refinement ทำต่อเนื่อง

Story / Backlog refinement
As a <role>, I need <function>, so that <value>; role ไม่ใช่ชื่อคน; มี description/details/assumptions/AC Given <precondition> When <event> Then <observable outcome>; And/But เสริม; ทดสอบบวก/ลบ ไม่กำหนด implementation เกินความต้องการ **INVEST**: Independent, Negotiable, Valuable, Estimable, Small, Testable; **Epic**: ใหญ่เกิน sprint → แบ่งดูรายการ/สร้าง/แก้ที่ละ vertical slice **Groom**: triage → reject/icebox/backlog; PO เรียงคุณค่าแยก/ชี้แจงงานล่วงหน้า 2 sprints; DoR คือพร้อมเริ่ม; ยังไม่กำหนดคน/sprint/estimate ถ้าไม่ได้ตกลง **Metadata**: Assignees, Labels (ชนิด), Milestone (sprint), Estimate (points); enhancement เพิ่มพฤติกรรม, bug แก้ผิด, technical debt ปรับโครง/refactor/env/dependencies

Sprint planning / execution
Points: effort/complexity/uncertainty แบบสัมพัทธ์; Fibonacci 1,2,3,5,8,13,21; anchor 3: +1 ขึ้น = 5, +2 = 8, −1 = 2; ไม่ใช่ชั่วโมง/ไม่เทียบข้ามทีม **Velocity**: Done points/sprint ของทีม; ใช้ประวัติและ capacity จริง; เลือกตาม goal, priority, dependencies และ ∑points ≤ capacity; ไม่เติมงานเพียงเพราะเหลือที่ **Planning**: PO ให้ goal/ลำดับ; ทีมประเมินและเลือกงาน/วิธีทำ; SM คุมเวลา; sprint backlog = forecast + plan; มักยาว 2 สัปดาห์ **Board**: New → Icebox/Backlog → Sprint Backlog → In Progress → Review/QA → Done; WIP 1/คน เว้นติดขัด; เลือกงานสำคัญสุด; เสร็จ → PR → merge/Done ตามกติกา **Daily**: 15 นาที คุยเมื่อวาน/วันนี้/อุปสรรค; ลงรายละเอียดหลังประชุม; SM ช่วยแก้สิ่งติดขัด **Review**: demo increment/AC กับ PO/ผู้เกี่ยวข้อง; feedback → Issue ใหม่. **Retro**: ทีม/SM ทบทวนสิ่งที่ดี/ไม่ดี/ต้องเปลี่ยน → action + owner + due; ตรวจผล sprint หน้า **Burndown**: x = เวลา, y = points คงเหลือ; ลดเมื่อ Done ไม่ใช่เริ่มทำ; metrics ต้องมี baseline และนำไปปรับงานได้ **จบ sprint (Lecture 7)**: งานไม่เริ่มคืน backlog/ถอนผู้รับ; งานค้างแยก story ส่วนที่เหลือตามสไลด์; ปิด/เปิด milestone; ทำงานส่วน ≠ ผ่าน DoD **Docs/maintenance**: req/design/technical/QA/user/SOP; แก้ bug/UI เพิ่ม feature/ปรับ req. **Org**: traditional หัวหน้าเดียว; matrix หลายสาย; Conway: โครงทีมสะท้อนโครงระบบ

Metrics: ความหมาย ไม่ใช่แค่ชื่อ

Metric	ใช้ตัดสินใจอะไร
Mean Lead Time	เวลา idea → production; ยาว → หาเวลารอ/คอขวด
Release frequency	ส่งขึ้นใช้งานบ่อยแค่ไหน; batch ใหญ่ทำให้ feedback ช้า
Change failure rate	สัดส่วน release/change ที่ก่อปัญหา; ไม่ใช่จำนวน tests ที่ผ่าน
MTTR	เวลาเฉลี่ยฟื้นระบบหลัง failure; ดูทั้งตรวจพบและกู้คืน

วัดอย่างมีบริบท: ระบุจุดเริ่ม/จบ หน่วย ช่วงเวลาและ baseline; commits/lines of code ≠ คุณค่าที่ส่งมอบ

วิศิษฐ์ สุวรรณเนาว์ 67070501042

Practice Midterm · Part 1

Requirement analysis · 25 คะแนน: ตอบ 5 ชั้น
Classify → purpose → quality + reason → check + expected → revise/justify; แยกข้อเท็จจริงจากข้อเสนอ

Workshop req	แนววิเคราะห์
Browse open	FR; Student ที่ login แล้วดู workshop ที่เปิดรับ; เตรียม open/closed → เห็นเฉพาะ open
“Quick”	NFR; กำกวม ไม่มี metric/load; เสนอ p95 ≤ 2 s ที่ 100 concurrent users รอลูกค้ายืนยัน
≤3/semester	BR/domain; ทดสอบนี้ 2 เพิ่มเป็น 3 ได้; มี 3 แล้วเพิ่มครั้งที่ 4 ต้องปฏิเสธ
Cancel + list	สอง FR ไม่ atomic; แยก Student ยกเลิกของตน/Coordinator ดูผู้สมัคร; ตรวจ role/owner ผ่าน API

NFR rewrite (เสนอ): Under 100 concurrent users, 95% of valid registration requests receive confirmation within 2 s. ถามจุดเริ่ม/จบการวัด network ปริมาณข้อมูลและกรณีล้มเหลว **FR template**: The system shall allow <actor> to <action/object> when <condition>. ระบุผลและข้อยกเว้นที่ตรวจได้

> quick/easy/reliable ยังไม่พอ; อย่าตั้งตัวเลขแล้วอ้างว่าลูกค้าให้มา; test ต้องมี input/state, action, expected result ไม่ใช่แค่ “ทำ unit test”

2) WorkshopHub proposal: 8 ช่อง

Element	หลักฐาน / สิ่งที่ขาด
Problem	หลายฟอร์ม รวบรวมด้วยมือ → ข้อมูลซ้ำ/ล่าช้า
Users	ระบุ Student/Coordinator; ต้องถามผู้ออนุมัติ
Solution	เว็บดู workshop/สมัคร/จัดการผู้เข้าร่วม
Value	ลดข้อผิดพลาด/งานธุรการ; ขาด metric/baseline
Scope	รุ่นแรกสมัครฟรี; payment/certificates ไว้ภายหลัง
Risk	ไม่ระบุ; เสนอเรื่องซ้ำซ้อน/สมัครพร้อมกัน/privacy
Effort	“ก่อนภาคหน้า” เป็น deadline ไม่ใช่ effort estimate
Evidence	ยังไม่มี AC/test/demo/sign-off

ย่อหน้าแก้ไข: ปัญหา/ผู้ใช้ → solution/ขอบเขตรุ่นแรก → คุณค่า/metric ที่เสนอ → risk/วิธีรับมือ → effort/ทรัพยากร/deadline → หลักฐาน/ผู้ยอมรับ เสนอให้ Coordinator ลองสมัคร/ยกเลิก ตรวจข้อมูลซ้ำและสิทธิ์; demo + tests ผ่าน + sign-off; ประเมิน person-days กับทีมก่อนรับปาก ไม่แต่งตัวเลขเป็นข้อเท็จจริง

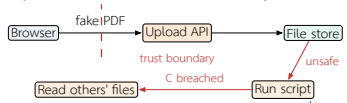
ตัวอย่าง revised proposal (ข้อเสนอเพิ่มเติม)
WorkshopHub will replace separate workshop forms that coordinators currently combine manually, causing duplicate registrations and delayed confirmations. Students will browse and register for free workshops; coordinators will manage participant lists. Payment and certificates are outside the first release.
We propose agreeing measurable error-reduction and administrative-effort targets with a baseline. Risks include concurrent duplicate registrations and privacy; verify server-side rules and role checks. Estimate effort after breaking down the work, then confirm feasibility before next semester. Completion should require traced acceptance tests, a working demo and coordinator sign-off. **ชี้แจงในคำตอบ**: ปัญหา/ผู้ใช้/ขอบเขตมาจากโจทย์; metric, risk/control และวิธีรับงานเป็นข้อเสนอ ไม่อ้างว่าลูกค้าตกลงแล้ว; deadline ไม่ใช่ effort

3) RBAC: default deny / ไม่สืบทอดสิทธิ์

Operation	Student	Coordinator
View published workshops	Allow	Allow
Submit own registration	Allow	Deny
Create workshop	Deny	Allow
Edit workshop	Deny	Allow
Export participant list	Deny	Allow

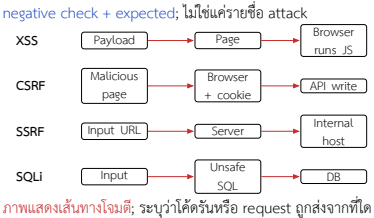
Coordinator ไม่ได้สิทธิ์สมัครเองโดยอัตโนมัติ; คนละ 1 role; ไม่ grant = deny; อย่าเพิ่มเงื่อนไข owner ที่โจทย์ไม่ให้ **Backend**: authenticate → role/operation → owner/state → validate → perform; คืน 401/403/404 ตามกติกา Negative test: Student เรียก `POST /workshops` ตรง → 403 และ DB ไม่เพิ่ม; A ขอ registration ของ B → ปฏิเสธ/ไม่รั่วข้อมูล

4) Upload attack: ข้าม trust boundary



Actor: นักศึกษาข้าม client check; **Assets**: รายงานผู้อื่น/server/DB; **Surface**: upload endpoint/file URL; **Boundary**: client → server, ข้อมูลอัปโหลด → โค้ดที่รัน **CIA**: อ่านรายงานคนอื่นและเมิด C; code execution เสี่ยง I/A ด้วย แยกเหตุที่เกิดแล้วจากผลที่อาจเกิด **Control/test 1**: server ตรวจ PDF type/content/size; ส่ง script, MIME ปลอมหรือไฟล์ใหญ่ → 4xx ไม่เก็บเป็นรายงาน **Control/test 2**: เก็บนอก webroot/รันไม่ได้; download handler ตรวจ owner; เปิด script → ไม่รัน; ขอรายงานคนอื่น → ปฏิเสธ เสริม: สุ่มชื่อไฟล์ ตรวจ path จำกัดสิทธิ์ สแกนมัลแวร์ตามความเสี่ยง และ log ปลอดภัย; frontend check ช่วย UX เท่านั้น

ตอบ security พร้อมเหตุผล
ช่องโหว่ → จุดเข้า/asset → boundary → CIA → defense → negative check + expected; ไม่ใช่เค้ารยชื่อ attack



ภาพแสดงเส้นทางโจมตี; ระบุว่าโค้ดรันหรือ request ถูกส่งจากที่ใด **Boundary**: จำกัด 3 → ทดสอบ 2/3/4, ข้ามภาค, ยกเลิกแล้วนับไหม? เงื่อนไขที่โจทย์ไม่บอกต้องถาม ไม่เพิ่มกฎเอง **แยกศัพท์**: threat = เหตุร้ายที่อาจเกิด; vulnerability = จุดอ่อน; attack = การใช้จุดอ่อน; risk = โอกาส × ผลกระทบ; control = มาตรการลดความเสี่ยง

Practice Midterm · Part 2

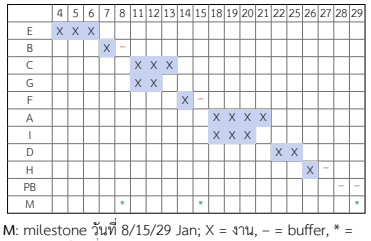
มุมมองตามข้อสอบ · 25 คะแนน: WBS/RACI/Gantt = PM; grooming = PO; planning = SM; แสดงเหตุผลตามบทบาท ไม่ใช่รับงานแทนทีม

1) DormFix: WBS 3 + RACI 2 + Gantt 5
WBS: 1.1 Requirements approved: E → B; 1.2 Designs approved: (C‖G) → F; 1.3 System accepted: (A‖I) → D → H; เว้น 1.4 ไม่เติม milestone เอง **Dependency**: E เขียน req → B อนุมัติ; C ออกแบบจอ/G ออกแบบ DB → F อนุมัติ; A ฟอร์ม/I dashboard → D ทดสอบทั้งคู่ → H ยอมรับ

Task	PM	Aom	Bank	Chai	Dao
E	A	R	C	I	–
C	A	C	R	I	–
G	A	–	C	R	I
D	A	I	–	C	R

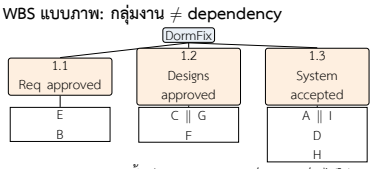
RACI: อิง meeting notes ในชุดฝึก 12 ข้อ ไม่เดาบทบาทจากชื่อตำแหน่ง **Gantt (Jan 2027)**: เริ่ม จ. 4; ทำเต็มเวลา จ.-ศ.; B/F/H รออนุมัติภายนอก เผื่อ task buffer 1 วัน; project buffer 2 วันท้าย

ID	Owner	d	Work dates (+buffer)
E	Aom	3	4-6 Jan
B	PM	1	7 Jan (+8 Jan)
C	Bank	3	11-13 Jan
G	Chai	2	11-12 Jan
F	PM	1	14 Jan (+15 Jan)
A	Chai	4	18-21 Jan
I	Dao	3	18-20 Jan
D	Dao	2	22,25 Jan
H	PM	1	26 Jan (+27 Jan)
PB	–	2	28-29 Jan



M: milestone วันที่ 8/15/29 Jan; X = งาน, – = buffer, * = milestone ไม่เพิ่มวันทำงาน; Start/Due นับรวม buffer; ลูกศร dependency ชี้งานที่รอ เหตุผล: B/F/H รอคนนอกที่มีงานอื่น; ทีมภายในใช้ buffer รวม 2 วันท้าย; C‖G และ A‖I ขนานได้; milestone ใช้เวลา 0 วัน

WBS แบบภาพ: กลุ่มงาน ≠ dependency



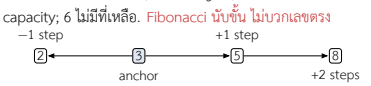
ตรวจ WBS: A-I ครบครั้งเดียว; milestone เป็นผลสำเร็จ ไม่ใช่งาน เพิ่ม; C/G และ A/I อยู่กลุ่มเดียวกันแต่ทำขนานได้; 1.4 ว่าง

2) Backlog grooming (10): Issue #12
As a student, I need to book a badminton court for one hour, so that I do not have to queue at the counter. **รายละเอียด**: มี courts/opening hours แล้ว; คนละ 1 booking/day; ตัวอย่าง Court 2 วันนี้ 17:00; ไม่เติม payment/cancel โดยไม่ถาม Given Court 2 is free at 17:00 today
When I book Court 2 at 17:00
Then Court 2 at 17:00 is shown as “Booked” **Negative AC**: Given มี booking วันนี้แล้ว, When ขอเพิ่มอีก, Then ปฏิเสธและไม่สร้าง booking ใหม่ **Metadata หลังกำหนดรายละเอียด**: Assignees –; Labels enhancement; Milestone –; Estimate –. ยังไม่ใช่ Sprint planning ไม่กำหนดคน/points/sprint เอง **Bug variant**: ยกเลิกแล้วยัง Booked → bug; Given ยกเลิกแล้ว, When เปิด slot, Then Available และจองใหม่ได้; คงข้อเท็จจริงจากลูกค้า **คำถามที่ควรถาม**: วัน/เวลาของการจองใช้เขตเวลาใด? ยกเลิกได้ไหมและคืนสิทธิ์วันนั้นหรือไม่? รายละเอียดที่ยังไม่ตกลงไม่เติมเป็น AC บังคับ

3) Sprint planning (5): Goal + capacity
ShuttleNow: goal = ติดตามรถ/เวลาถึง; anchor 3, velocity 16

#	Story / เทียบ anchor	Pt	Pick
1	Live position (+2 ชั้น)	8	Yes
2	QR fare (เท่ากัน)	3	No
3	Arrival at my stop (+1)	5	Yes
4	Arrival alert (เท่ากัน)	3	Yes
5	Arrival at all stops (+1)	5	No
6	Fix mobile truncation (−1)	2	No

เลือก 1/3/4: 8+5+3=16; 2 ไม่ตรง goal; เพิ่ม 5 รวม 21 เกิน capacity; 6 ไม่มีที่เหลือ. Fibonacci นับขั้น ไม่บวกเลขตรง



RACI check: A ชัดคนเดียว; R ลงมือ; C ปรึกษาสองทางก่อนตัดสินใจ; I แจ้งทางเดียว; dependency ≠ responsibility

PERT (Lecture 6): คำนวณ effort ไม่ใช่วันนัดส่ง
T1: O=2, M=3, P=6 h → E=3.33 h, SD=0.67 h. หกงานในสไลด์รวม E=35.67 h, SD=3.23 h เมื่อ estimates อิสระ → 35.7 ± 3.2 h. วันจบยังต้องดู dependency/CP/คน ไม่บวกงานขนานเป็น duration

Git: คำสั่ง remote ที่ควรรู้
`git fetch origin`
รับ commits/อัปเดต origin/*; ไม่เลื่อน local branch, HEAD หรือ working tree ในการ fetch ปกติ
`git switch main`
`git merge --ff-only origin/main`
อัปเดต main แบบ FF; ถ้าแยกสายจะปฏิเสธ ไม่สร้าง merge commit
`git push origin sprint_1_staging`
เผยแพร่ staging/อัปเดต tracking ref เมื่อสำเร็จ; push feature ไม่เผยแพร่ merge ที่อยู่เฉพาะ staging

Practice Midterm · Part 3

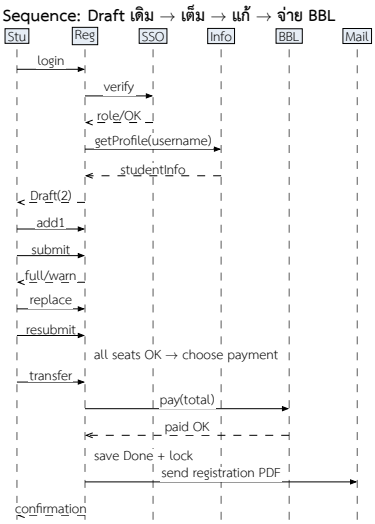
Blueprint: เลือก Sequence/Activity/UI/Use Case/State อย่างละ 10 รวม 30; ไม่จำเป็นต้องตรง 3 แบบใน mock

KU Registration: อ่าน R1-R11 ก่อนวาด
SSO คืน role; ใช้ภาคปัจจุบัน/Student Info; เลือกหลาย offering รวมยอด; Draft บันทึก/แก้ได้; ตรวจทุก section; เต็ม → แก้/ส่งใหม่; card → KBank, transfer → BBL; จ่ายสำเร็จ → Done/ล็อก/email PDF

Actors (R1-R4)	ขอบเขตที่ระบุในโจทย์
Department Admin	สร้าง course; เลือก semester/code แล้วกำหนด section/room/time/notes/max students
University Admin	สร้าง semester และกำหนด active semester

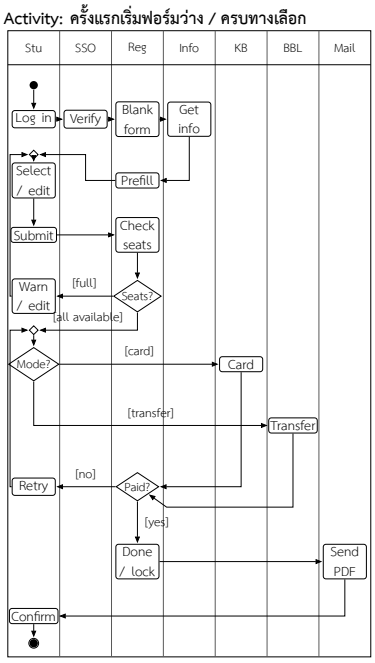
Instructor เป็น role ที่ SSO คืนได้ แต่ R1-R11 ไม่กำหนดงานให้; ไม่แต่งสิทธิ์เพิ่มเอง

Sequence: Draft เดิม → เต็ม → แก้ → จ่าย BBL



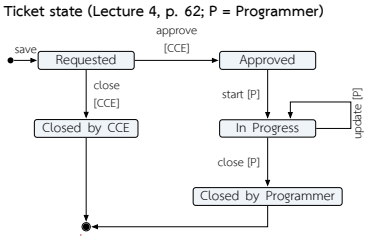
เริ่มจาก Draft เดิม 2 offerings แล้วเพิ่ม 1; ส่งแล้วเต็มก่อนแก้/ส่งซ้ำสำเร็จ; Reg จัดการ DB ภายใน; หากโจทย์ให้ DB lifeline ต้องแสดง call

Activity: ครั้งแรกเริ่มฟอร์มว่าง / ครบทางเลือก

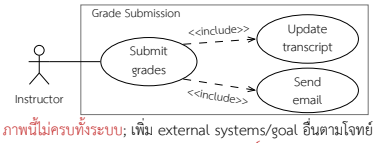


KB = KBank; BBL = Bangkok Bank; Info = Student Info; ใส่ guard/loop กลับจุดแก้; จ่ายไม่สำเร็จห้าม Done/ส่ง PDF

โจทย์ทางเลือก: Use Case / State
Ticket state (Lecture 4, p. 62; P = Programmer)



ปิดแล้วห้ามเพิ่ม action; save [System] → Requested; action บันทึกแล้วแก้/ลบไม่ได้ **Grades**: submit เฉพาะ Ready → Submitted; ปรับ transcript/ส่ง email (Use Case ย่อด้านล่าง)



ภาพนี้ไม่ครบทั้งระบบ; เพิ่ม external systems/goal อื่นตามโจทย์ Registration ≠ Grades; อย่าย้าย state/สิทธิ์ข้าม case **เช็ก UML**: decision แตกทางเลือกพร้อม guard; merge รวมทางเลือก; fork/join แตก/รอทางขนาน. Sequence ระบุผู้ส่ง/ผู้รับและผล; State ใช้ event [guard] / effect ของ object เดียว

Practice Midterm · Part 4

UI ต่อ / Git · 20 คะแนน

Course Offering List — Student

Semester: active ///// Course: [search]

Code	Sec	Cr	Fee	Seats
CPE334	1	3	8,000	3

Detail: ID, name, room, time, notes, semester, maximum, number registered
[Select / Add to Form] [Prev] 1 [Next]

Student Registration Form

[Close] [Save Draft] [Submit]
No.: auto ///// Semester: active /////
Student ID / name / username /////
Degree programme / email /////
Total credits / fee ///// Status: Draft /////

*Offering [LoV] | Code / Sec / Cr / Fee /////
113 [select] | CPE334 / 1 / 3 / 8,000

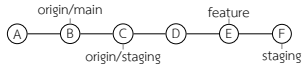
[+ Add line] [Remove line]
Full section → warning → edit / resubmit
Done → locked; confirmation + emailed PDF

///// : ขีดเฉียงแทน read-only; *Offering เป็น FK ที่แก้ได้ เลือกด้วย LoV; ข้อมูลที่ดึงมา/total/semester/student อ่านอย่างเดียว; แสดง R7 ครบผ่าน columns/detail ไม่ต้องยัดทุก field ลงตารางเดียว **ตรวจ UI**: filters/selection, header/line items, add/remove, total, state, permissions, validation; เต็ม → แก้/ลองใหม่; Done → แก้ไม่ได้

Git: graph = ancestry + pointers
HEAD → current branch → commit; เมื่อ clean จะได้ working tree ตาม HEAD; origin/* คือ remote-tracking ref สถานะ remote ที่ local รู้ล่าสุด ไม่ใช่ GitHub แบบสด; tag ชี้ commit คงที่ **switch**: เปลี่ยน HEAD/working tree ไม่เลื่อน branch; **commit**: สร้างลูกของ HEAD แล้วเลื่อน branch ปัจจุบัน; **merge**: รวมเข้า branch ปัจจุบัน ไม่เลื่อน source **FF**: current เป็น ancestor ของ target → เลื่อน current ไม่สร้าง commit; **3-way**: แยกสาย → commit ใหม่ 2 parents; graph ไม่บอกว่าจะมี file conflict

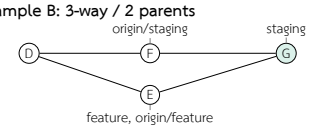
Example A: FF → new commit → switch
เริ่ม main=B, staging=C, feature=E บน C → D → E; origin/main=B, origin/staging=C

```
git switch sprint_1_staging
git merge feature/load_scores
git add score_data.py scores.csv
git add validation_notes.txt
git commit -m \
"Document score validation cases"
git switch main
```



ผล: FF C → E; F อยู่บน staging, feature ยัง E, origins เดิม; switch ท้าย → HEAD → main=B

Example B: 3-way / 2 parents



`git merge feature/basic_statistics` บน F → G มี parents F/E; switch main ท้าย → HEAD=B; ภาพขยาย fork ที่ D; main/origin/main=B เป็น ancestor ของ D; source/origins ไม่เลื่อน

Sample 2.8: สร้าง feature จากฐานหลัง FF
หลัง FF จาก C → E บน staging จึงสร้าง branch ใหม่ที่ E; `branch` อย่างเดียวยังไม่เปลี่ยน HEAD

```
git branch feature/grade_distribution
git switch feature/grade_distribution
```

แก้ `report.py`, `scores.csv`; สร้าง `grade_ranges.csv` แล้ว stage เฉพาะสามไฟล์/commit F → HEAD อยู่ feature ใหม่=F; staging และ feature/pass_rate ยัง E, origin/staging ยัง C **Inspect**: `status` ดู untracked/index; `diff` ดู unstaged; `diff --cached` ดู staged; sample ห้ามใช้ `git add .` **วาดให้ครบ**: commits/parents, branch/origin ทุกตัว, tag, HEAD; merge มี 2 parents; commit ที่มีเฉพาะ local คือส่วนที่ ahead จาก tracking ref

Explain state: fetch / tag / unpublished merge
Sample 2.6: A → B → H; main=B, origin/main=v1.0=H; switch main ได้ B; FF origin/main ได้ H; staging/feature/tag ไม่เลื่อน **Sample 2.7**: staging=H (parents E/G), origin/staging=E, feature=G; push feature ไม่เผยแพร่ H; ต้อง `git push origin sprint_1_staging` → origin/staging=H เมื่อสำเร็จ

Sample 2.9: อัปเดต staging ก่อนรวม feature
เริ่ม staging=D, origin/staging=G; D → E → F อยู่บน feature และ D → G แยกอีกสาย

```
git switch sprint_2_staging
git merge --ff-only \
origin/sprint_2_staging
git merge feature/export_results
git push origin sprint_2_staging
```

วาดให้ละชั้น: switch → HEAD ที่ D; FF → staging=G; merge → H มี parents G/F; push → origin/staging=H; feature ยัง F และ main ไม่เลื่อน ห้าม FF feature ที่แยกสาย; FF ใช้เฉพาะ D → G; ไม่รวม remote ที่ยังไม่ได้ fetch โดยเดาจาก GitHub

วิศิษฐ์ สุวรรณเนาว์ 67070501042